PRENOM ET NOM : Sidy Ababacar NOYE

SUJET1

1. Analyse et Modélisation UML

A. Cas d'Utilisation (Use Cases)

L'enseignant est l'acteur principal qui gère le système.

- **Acteur :** Enseignant.
- **Système :** Application de gestion de projets.
- **Cas d'utilisation :**
    - Créer une formation.
    - Inscrire un étudiant.
    - Créer un projet.
    - Affecter des étudiants à un binôme.
    - Saisir les notes (Rapport et Soutenance).
    - Calculer la note finale (incluant le calcul du retard).

B. Diagramme de Classes

Voici la structure logique basée sur l'énoncé :

- **Formation :** possède plusieurs Étudiants.
- **Étudiant :** appartient à une Formation et participe à des Binômes.
- **Projet :** définit le sujet et la date limite.
- **Binôme :** lié à un Projet, contient 1 à 2 Étudiants, et stocke la note de rapport ainsi que la date de remise effective.

Extrait de code

```
classDiagram
  class Formation {
    - _id_formation: int
```

```
    - _nom: str
    - _promotion: str
    + __init__(id, nom, promo)
    + nom() str
}

class Etudiant {
    - _num_etudiant: int
    - _nom: str
    - _prenom: str
    - _notes_soutenance: dict
    + __init__(num, nom, prenom)
    + ajouter_note_soutenance(id_projet, note)
    + obtenir_note_soutenance(id_projet) float
}

class Projet {
    - _id_projet: int
    - _matiere: str
    - _sujet: str
    - _date_limite: date
    + __init__(id, matiere, sujet, date_limite)
    + date_limite() date
}

class Binome {
```

```
    - _id_relatif: int
    - _date_remise_effective: date
    - _note_rapport: float
    + __init__(id_relatif, projet, etudiants)
    + calculer_note_finale(etudiant) float
  }

  Formation "1" --> "*" Etudiant : contient
  Etudiant "1..2" -- "*" Binome : compose
  Projet "1" -- "*" Binome : est réalisé par
```

C. Diagramme de Séquence : Calcul de la note finale

Ce scénario montre comment le système calcule la note d'un étudiant en tenant compte du retard.

1. L'Enseignant demande la note finale d'un Etudiant pour un Projet via l'objet Binome.
2. Binome récupère la date_limite du Projet.
3. Binome compare avec sa date_remise_effective pour calculer les points de pénalité.
4. Binome récupère la note de soutenance individuelle de l' Etudiant.
5. Binome combine la note de rapport, la soutenance et applique le malus.
6. Le résultat est retourné à l'Enseignant.

2. Implémentation Python (POO)

Le code suivant applique l'encapsulation, les propriétés et la logique métier demandée.

Python

```
from datetime import datetime, date
class Formation:
  def __init__(self, id_formation, nom, promotion):
```

```
        self._id_formation = id_formation
        self._nom = nom
        self._promotion = promotion

    @property
    def nom(self):
        return self._nom

    def __str__(self):
        return f"Formation {self._nom} ({self._promotion})"

class Etudiant:
    def __init__(self, num_etudiant, nom, prenom):
        self._num_etudiant = num_etudiant
        self._nom = nom
        self._prenom = prenom
        # Stockage des notes de soutenance par ID de projet
        self._notes_soutenance = {}

    @property
    def nom_complet(self):
        return f"{self._prenom} {self._nom}"

    def ajouter_note_soutenance(self, id_projet, note):
        self._notes_soutenance[id_projet] = note
```

```
    def obtenir_note_soutenance(self, id_projet):
        return self._notes_soutenance.get(id_projet, 0)

    def __repr__(self):
        return f"Etudiant({self._num_etudiant}, {self._nom})"

class Projet:
    def __init__(self, id_projet, matiere, sujet, date_limite):
        self._id_projet = id_projet
        self._matiere = matiere
        self._sujet = sujet
        self._date_limite = date_limite # Objet date

    @property
    def id_projet(self):
        return self._id_projet

    @property
    def date_limite(self):
        return self._date_limite

class Binome:
    def __init__(self, id_relatif, projet, etudiants, date_remise_effective, note_rapport):
        self._id_relatif = id_relatif
        self._projet = projet
        self._etudiants = etudiants # Liste de 1 ou 2 étudiants
```

```
        self._date_remise_effective = date_remise_effective
        self._note_rapport = note_rapport

    def calculer_retard(self):
        """Calcule le nombre de jours de retard[cite: 29]."""
        diff = (self._date_remise_effective - self._projet.date_limite).days
        return max(0, diff)

    def calculer_note_finale(self, etudiant):
        """Calcule la note finale pour un étudiant précis."""
        if etudiant not in self._etudiants:
            return 0

        note_soutenance = etudiant.obtenir_note_soutenance(self._projet.id_projet)
        points_retard = self.calculer_retard() * 1 # Exemple: -1 point par jour

        # Formule arbitraire : moyenne rapport/soutenance - retard
        note_finale = ((self._note_rapport + note_soutenance) / 2) - points_retard
        return max(0, note_finale)

# --- Programme Principal ---
if __name__ == "__main__":
    # 1. Création de la formation
    m1_id = Formation(1, "MIAGE-ID", "Initiale")

    # 2. Création des étudiants
```

```
e1 = Etudiant(101, "Dupont", "Jean")
e2 = Etudiant(102, "Lefebvre", "Marie")

# 3. Création d'un projet
projet_uml = Projet(1, "UML", "Gestion de Projets", date(2026, 4, 10))

# 4. Saisie des notes de soutenance (individuelles)
e1.ajouter_note_soutenance(1, 15)
e2.ajouter_note_soutenance(1, 18)

# 5. Création du binôme et remise du rapport (avec 2 jours de retard)
# Remis le 12 avril au lieu du 10 avril
binome1 = Binome(1, projet_uml, [e1, e2], date(2026, 4, 12), 16)

# 6. Affichage des résultats
print(f"--- Résultats Projet : {projet_uml._sujet} ---")
for etu in [e1, e2]:
    note = binome1.calculer_note_finale(etu)
    print(f"Etudiant: {etu.nom_complet} | Note Finale: {note}/20 (Retard: {binome1.calculer_retard()} jours)")
```

Sujet 2

Diagramme de classes (description claire)

Classes principales :

Eleve

- id
- nom
- prénom
- adresse
- date_naissance
- séances (liste)
- examens

Seance

- date
- heure
- série utilisée
- résultats (élèves + fautes)

CDRom

- id
- éditeur
- séries (6)

Serie

- numéro
- questions (40)

- cdrom

Question

- intitulé
- réponse
- difficulté
- thème

Participation

(relation élève ↔ séance)

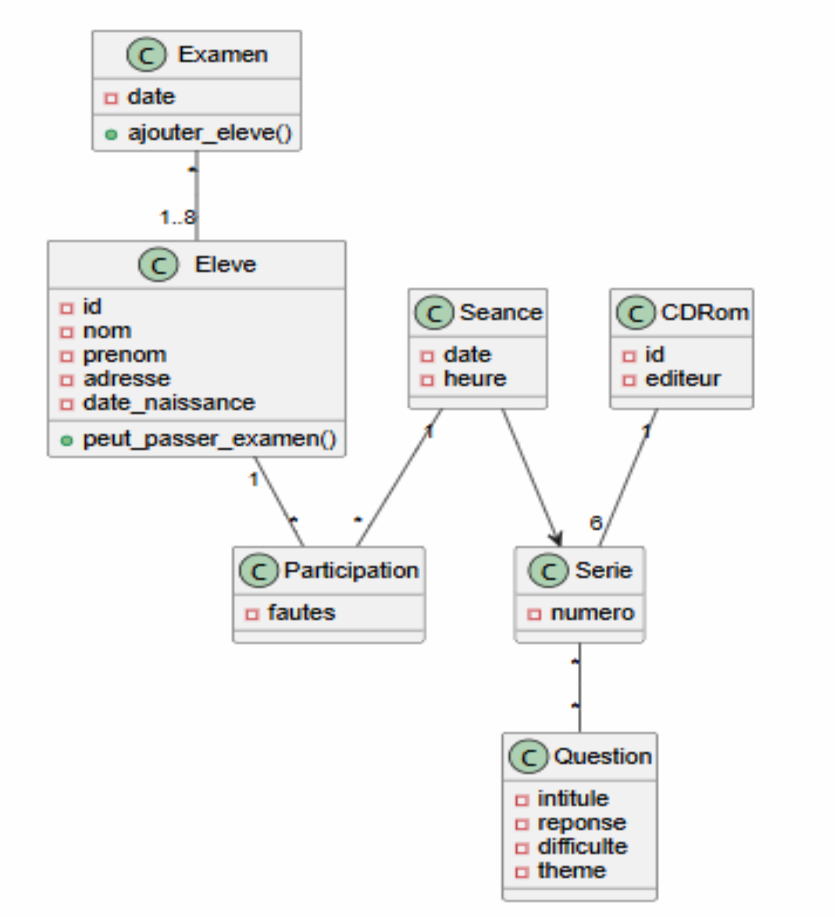
- élève
- séance
- fautes

Examen

- date
- élèves (max 8)
- résultats (fautes)

Relations UML

- Élève 1 —— * Participation
- Séance 1 —— * Participation
- CD-ROM 1 —— 6 Séries
- Série * —— * Questions
- Séance 1 —— 1 Série
- Examen 1 —— max 8 Élèves



Cas d'utilisation

Acteurs :

- Directeur
- Élève

Cas :

- Ajouter élève
- Créer séance
- Choisir série
- Enregistrer fautes
- Vérifier admissibilité examen
- Planifier examen

- Enregistrer résultat examen
- Consulter statistiques

Diagramme de séquence

Scénario : Autoriser élève à passer examen

1. Directeur → Élève : vérifier admissibilité
2. Élève :
    - récupère 4 dernières participations
    - vérifie fautes ≤ 5
3. Retour : OUI / NON

Implémentation Python

```
from datetime import datetime


class Question:
    def __init__(self, intitule, reponse, difficulte, theme):
        self.__intitule = intitule
        self.__reponse = reponse
        self.__difficulte = difficulte
        self.__theme = theme



class Serie:
    def __init__(self, numero, cdrom):
        self.__numero = numero
        self.__cdrom = cdrom
        self.__questions = []
```

```
    def ajouter_question(self, question):
        self.__questions.append(question)


class CDRom:
    def __init__(self, id_cd, editeur):
        self.__id = id_cd
        self.__editeur = editeur
        self.__series = []

    def ajouter_serie(self, serie):
        self.__series.append(serie)


class Eleve:
    def __init__(self, id_e, nom, prenom, adresse, date_naissance):
        self.__id = id_e
        self.__nom = nom
        self.__prenom = prenom
        self.__adresse = adresse
        self.__date_naissance = date_naissance
        self.__participations = []
        self.__examens = []

    def ajouter_participation(self, participation):
```

```
        self.__participations.append(participation)

    def peut_passer_examen(self):
        if len(self.__participations) < 4:
            return False

        dernieres = self.__participations[-4:]
        return all(p.fautes <= 5 for p in dernieres)

    def __str__(self):
        return f"{self.__nom} {self.__prenom}"


class Seance:
    def __init__(self, date, heure, serie):
        self.__date = date
        self.__heure = heure
        self.__serie = serie
        self.__participations = []

    def ajouter_participation(self, participation):
        self.__participations.append(participation)


class Participation:
    def __init__(self, eleve, seance, fautes):
```

```
        self.__eleve = eleve
        self.__seance = seance
        self.__fautes = fautes

        eleve.ajouter_participation(self)
        seance.ajouter_participation(self)

    @property
    def fautes(self):
        return self.__fautes



class Examen:
    def __init__(self, date):
        self.__date = date
        self.__eleves = []
        self.__resultats = {}

    def ajouter_eleve(self, eleve):
        if len(self.__eleves) >= 8:
            raise Exception("Max 8 élèves")

        if not eleve.peut_passer_examen():
            raise Exception("Élève non autorisé")

        self.__eleves.append(eleve)
```

```
    def enregistrer_resultat(self, eleve, fautes):
        self.__resultats[eleve] = fautes

    def eleves_recales(self):
        return [e for e, f in self.__resultats.items() if f > 5]



# TEST
if __name__ == "__main__":
    cd = CDRom(1, "Code Rousseau")
    serie = Serie(1, cd)

    e = Eleve(1, "Diop", "Ali", "Dakar", "2000-01-01")

    s = Seance("2026-04-01", "10:00", serie)

    Participation(e, s, 4)
    Participation(e, s, 3)
    Participation(e, s, 5)
    Participation(e, s, 2)

    print("Peut passer examen ?", e.peut_passer_examen())

    exam = Examen("2026-04-20")
    exam.ajouter_eleve(e)
```

```
exam.enregistrer_resultat(e, 6)

print("Recalés :", exam.eleves_recales())
```

SUJET3

Diagramme de classes (description claire)

Classes principales

Auteur

- nom
- prénom
- num_carte_pro

Revue

- nom
- redacteur_chef (Auteur)
- numeros

Numero

- numero
- annee
- revue

- articles

Article

- titre
- contenu
- auteurs

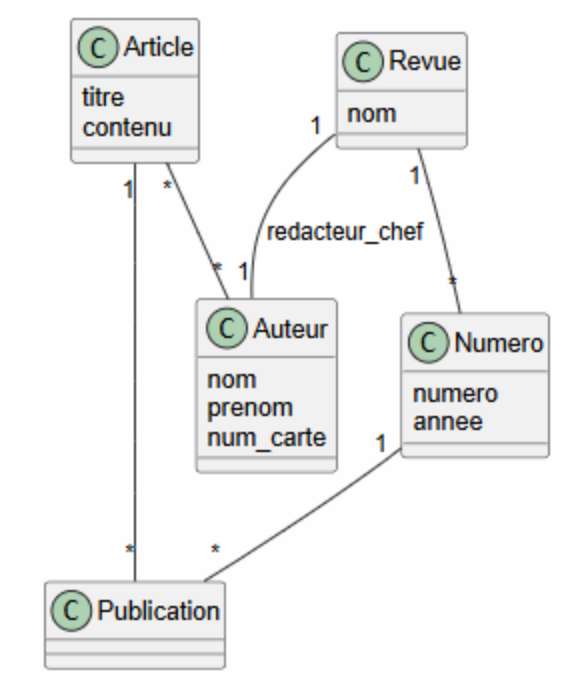
Classe intermédiaire Publication

pour gérer :

- Article ↔ Revue ↔ Numéro
- article
- numero

📎 **Relations UML**

- Revue 1 —— * Numero
- Numero 1 —— * Publication
- Article 1 —— * Publication
- Article * —— * Auteur
- Revue 1 —— 1 Auteur (rédacteur)



Cas d'utilisation

Acteurs :

- Administrateur
- Auteur

Cas :

- Créer revue
- Ajouter auteur
- Nommer rédacteur en chef
- Créer numéro
- Ajouter article
- Associer article à numéro
- Consulter articles
- Rechercher articles multi-revues

Diagramme de séquence

Scénario : Ajouter un article dans un numéro

1. Admin → Numero : ajouter article
2. Numero → Publication : créer
3. Publication → Article : associer
4. Retour OK

Code Python complet

```
class Auteur:
    def __init__(self, nom, prenom, num_carte):
        self.__nom = nom
        self.__prenom = prenom
        self.__num_carte = num_carte

    def __str__(self):
        return f"{self.__nom} {self.__prenom}"


class Article:
    def __init__(self, titre, contenu):
        self.__titre = titre
        self.__contenu = contenu
        self.__auteurs = []

    def ajouter_auteur(self, auteur):
        self.__auteurs.append(auteur)
```

```
    def __str__(self):
        return self.__titre


class Revue:
    def __init__(self, nom, redacteur_chef):
        self.__nom = nom
        self.__redacteur_chef = redacteur_chef
        self.__numeros = []

    def ajouter_numero(self, numero):
        self.__numeros.append(numero)

    def __str__(self):
        return self.__nom


class Numero:
    def __init__(self, numero, annee, revue):
        self.__numero = numero
        self.__annee = annee
        self.__revue = revue
        self.__publications = []

        revue.ajouter_numero(self)
```

```
    def ajouter_article(self, article):
        pub = Publication(article, self)
        self.__publications.append(pub)

    def get_articles(self):
        return [p.article for p in self.__publications]


class Publication:
    def __init__(self, article, numero):
        self.__article = article
        self.__numero = numero

    @property
    def article(self):
        return self.__article


# TEST
if __name__ == "__main__":
    a1 = Auteur("Diop", "Ali", "123")
    a2 = Auteur("Sow", "Awa", "456")

    revue = Revue("Linux Magazine", a1)

    num1 = Numero(1, 2026, revue)
```

```
art1 = Article("Python Avancé", "Contenu...")
art1.ajouter_auteur(a1)
art1.ajouter_auteur(a2)

num1.ajouter_article(art1)

print("Articles du numéro :", num1.get_articles())
```

SUJET 4

Diagramme de classes (description claire)

Classes principales

Personne

- nom
- prénom

Assure (hérite de Personne)

- numéro_assuré

Medecin (hérite de Personne)

- numéro_medecin

Generaliste (hérite de Medecin)

Specialiste (hérite de Medecin)

Patient

- assure
- medecin_traitant (optionnel)

Consultation

- date
- patient
- medecin_generaliste
- prescriptions

Prescription

- type (médicament / spécialiste)

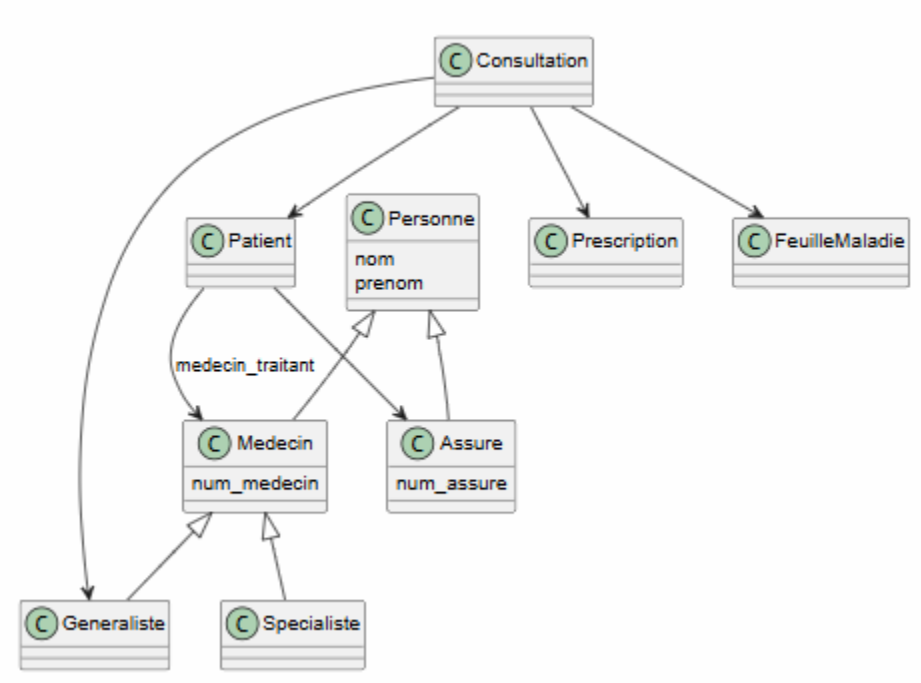
FeuilleMaladie

- consultation
- montant

Relations UML

- Personne ⟶ Assure
- Personne ⟶ Medecin
- Medecin ⟶ Generaliste / Specialiste
- Patient —— 1 Assure
- Patient —— 0..1 Medecin traitant
- Consultation —— 1 Patient

- Consultation —— 1 Generaliste
- Consultation —— * Prescription
- Consultation —— 1 FeuilleMaladie



Cas d'utilisation

Acteurs :

- Assuré
- Médecin
- Sécurité sociale

Cas :

- S'inscrire (assuré)
- Choisir médecin traitant
- Consulter médecin
- Prescrire médicaments
- Prescrire spécialiste

- Enregistrer feuille maladie
- Effectuer remboursement

Diagramme de séquence

Scénario : Consultation + remboursement

1. Patient → Généraliste : consultation
2. Généraliste → Consultation : créer
3. Généraliste → Prescription : ajouter
4. Consultation → FeuilleMaladie : créer
5. Sécurité sociale → FeuilleMaladie : rembourser

CODE PYTHON

```
class Personne:
    def __init__(self, nom, prenom):
        self.__nom = nom
        self.__prenom = prenom

    def __str__(self):
        return f"{self.__nom} {self.__prenom}"


class Assure(Personne):
    def __init__(self, nom, prenom, num_assure):
        super().__init__(nom, prenom)
        self.__num_assure = num_assure
```

```
class Medecin(Personne):
    def __init__(self, nom, prenom, num_medecin):
        super().__init__(nom, prenom)
        self.__num_medecin = num_medecin


class Generaliste(Medecin):
    pass


class Specialiste(Medecin):
    pass


class Patient:
    def __init__(self, assure):
        self.__assure = assure
        self.__medecin_traitant = None

    def choisir_medecin_traitant(self, medecin):
        self.__medecin_traitant = medecin


class Prescription:
    def __init__(self, type_prescription, details):
        self.__type = type_prescription
```

```
        self.__details = details



class Consultation:
    def __init__(self, patient, medecin, date):
        self.__patient = patient
        self.__medecin = medecin
        self.__date = date
        self.__prescriptions = []

    def ajouter_prescription(self, prescription):
        self.__prescriptions.append(prescription)



class FeuilleMaladie:
    def __init__(self, consultation, montant):
        self.__consultation = consultation
        self.__montant = montant

    def rembourser(self, taux=0.7):
        return self.__montant * taux



# TEST
if __name__ == "__main__":
    assure = Assure("Diop", "Ali", "A123")
```

```
patient = Patient(assure)

med = Generaliste("Sow", "Fatou", "M456")
patient.choisir_medecin_traitant(med)

cons = Consultation(patient, med, "2026-04-15")

cons.ajouter_prescription(Prescription("medicament", "Paracetamol"))
cons.ajouter_prescription(Prescription("specialiste", "Cardiologue"))

feuille = FeuilleMaladie(cons, 10000)

print("Remboursement :", feuille.rembourser())
```

Sujet 5

Diagramme de classes (description)

Classes principales

Enseignant

- numen
- nom
- prénom

Professeur (hérite de Enseignant)

- hdr

MCF (hérite de Enseignant)

- these

Laboratoire

- nom
- directeur (Professeur)

Master

- nom
- directeur (Enseignant)
- cours

Cours

- intitulé

Classe intermédiaire Affectation

pour gérer :

- obligatoire / optionnel
- cours
- master
- obligatoire (bool)

Relations UML

- Enseignant ⟶ Professeur / MCF
- Professeur —— 0..1 Laboratoire

- Master —— 1 Enseignant
- Master —— * Affectation
- Cours —— * Affectation

Cas d'utilisation

Acteurs :

- Administrateur

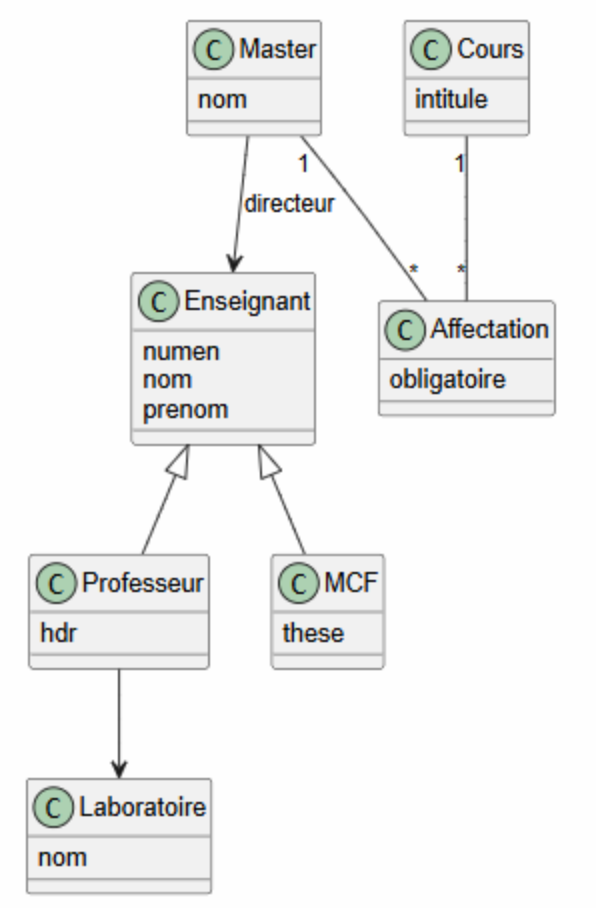
Cas :

- Ajouter enseignant
- Spécialiser enseignant
- Créer master
- Affecter directeur
- Ajouter cours
- Définir cours obligatoire / optionnel
- Associer cours à master

Diagramme de séquence

Scénario : Ajouter un cours à un master

1. Admin → Master : ajouter cours
2. Master → Affectation : créer
3. Affectation → Cours : associer
4. Retour OK



CODE PYTHON

```
class Enseignant:
    def __init__(self, numen, nom, prenom):
        self.__numen = numen
        self.__nom = nom
        self.__prenom = prenom

    def __str__(self):
        return f"{self.__nom} {self.__prenom}"


class Professeur(Enseignant):
```

```
    def __init__(self, numen, nom, prenom, hdr):
        super().__init__(numen, nom, prenom)
        self.__hdr = hdr
        self.__laboratoire = None

    def diriger_labo(self, labo):
        self.__laboratoire = labo


class MCF(Enseignant):
    def __init__(self, numen, nom, prenom, these):
        super().__init__(numen, nom, prenom)
        self.__these = these


class Laboratoire:
    def __init__(self, nom, directeur):
        self.__nom = nom
        self.__directeur = directeur
        directeur.diriger_labo(self)


class Cours:
    def __init__(self, intitule):
        self.__intitule = intitule
```

```
    def __str__(self):
        return self.__intitule



class Affectation:
    def __init__(self, cours, master, obligatoire):
        self.__cours = cours
        self.__master = master
        self.__obligatoire = obligatoire



class Master:
    def __init__(self, nom, directeur):
        self.__nom = nom
        self.__directeur = directeur
        self.__affectations = []

    def ajouter_cours(self, cours, obligatoire=True):
        aff = Affectation(cours, self, obligatoire)
        self.__affectations.append(aff)

    def afficher_cours(self):
        for aff in self.__affectations:
            type_cours = "Obligatoire" if aff._Affectation__obligatoire else "Optionnel"
            print(aff._Affectation__cours, "-", type_cours)
```

```
# TEST
if __name__ == "__main__":
    prof = Professeur("N123", "Diop", "Ali", "HDR Info")
    labo = Laboratoire("Labo IA", prof)

    master = Master("Master IA", prof)

    c1 = Cours("Machine Learning")
    c2 = Cours("Big Data")

    master.ajouter_cours(c1, True)
    master.ajouter_cours(c2, False)

    master.afficher_cours()
```